MixBytes()

[SECURITY AUDIT REPORT]

Lido Oracle

PREPARED FOR LIDO

REPORT DATE

July 26, 2026

[CONTENTS]

Table of Contents

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The Lido Oracle is an off-chain daemon for the Lido decentralized staking protocol that observes state across the Execution and Consensus layers and submits periodic reports to Lido's on-chain contracts through a hash-consensus flow among a quorum of oracle members. This engagement covered a targeted change set that replaces the BLS signature verification library used for deposit proof-of-possession checks and adds structured logging across the pending-deposit accounting path. The verification routine gates whether pending deposits are attributed to Lido when computing `clPendingBalance`, making it a correctness-critical component of the accounting module.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

Our review focused on the core verification logic and its interaction with the vendored native cryptographic dependency, since the change introduces compiled C code into a process that also holds oracle quorum signing keys. Alongside the project-specific analysis, the team worked through the standard internal checklist covering input validation, exception handling, secret management, dependency supply chain, and failure modes under adversarial input. Beyond the general checklist, the following areas were investigated in depth:

- **BLS verifier equivalence.** Both libraries were built and loaded into a single interpreter, and the audited call pattern was executed against identical inputs across valid signatures, well-formed but incorrect signatures, points at infinity, all-zero and off-curve encodings, low-order subgroup points, and truncated inputs. No divergence in verdicts was observed.
- **Exception surface of the SWIG bindings.** The generated bindings were compiled and probed to enumerate the exception types actually raised, rather than relying on the declared interface, in order to confirm that the `try` / `except` clause fully covers the negative path and cannot allow an exception to escape into the accounting cycle.
- **Private key isolation.** The verification module's import graph, the runtime environment-variable masking, and the compiled extension's dynamic dependencies and imported symbols were examined to establish whether the new dependency can reach quorum signing material.
- **Dependency provenance.** The submodule pin was resolved against upstream release tags, and the vendored sources were scanned for network, filesystem, and process-execution primitives.
- **Version stability assessment.** The full delta between the upstream `v0.3.16` and `v0.3.17` releases was reviewed, and the audited test suite was executed against both builds to determine whether downgrading to the more established release is safe.
- **Accounting logging correctness.** The added log statements were checked against the values they report.

The vendored `blst` library (https://github.com/supranational/blst) was not subject to a line-by-line cryptographic review as part of this engagement; our assessment covered its integration surface, build pipeline, exception behaviour, and provenance, drawing on the library's public release history and its established use across Ethereum consensus clients.

This engagement is not a review of a fix for the pending-balance accounting discrepancy, which remains under investigation by the Lido team. The change set covers a library substitution and the addition of diagnostic logging; it does not modify the accounting algorithm, and the root cause of the discrepancy had not been established at the time of review. Any remediation that follows from that investigation falls outside this scope and would require a separate review.

In particular, the change set does not address the suspected race condition in the interaction with the Keys API. Its contribution is twofold: the replacement of the pure-Python BLS implementation with a compiled one materially reduces the time required to assemble a report, and the added per-stage instrumentation records the intermediate values needed to observe the race when it next occurs. Faster assembly narrows the window in which the oracle's view of the key set can drift from the on-chain state, while the logging is what allows a subsequent divergence to be attributed to a specific stage of the pending-deposit path rather than reconstructed after the fact.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The change set is sound overall and no issues affecting protocol security were identified.** The diff is confined to a library substitution plus additive logging. Outside the verifier swap, no behavioural change was introduced into the accounting path - the remaining edits hoist expressions into named variables and add counters. Differential testing confirmed the two libraries agree on every input class exercised.
- **Migrating to `v0.3.16` is supported by our review.** The delta between the two upstream releases is dominated by removal of duplicated assembly, new language bindings unrelated to Python, and fixes in Go/Rust/Java bindings. The core verification path - `pairing.c`, `e1.c`, `e2.c`, `map_to_g1.c`, `map_to_g2.c`, `hash_to_field.c` - is unchanged between the two tags. The audited test suite passes against both builds, exception types are identical, and the performance difference is immaterial relative to the improvement over the previous library. The more established release carries a longer production track record.
- **Key handling remains correctly separated.** The verification module does not import the variables module and has no visibility of `MEMBER_PRIV_KEY`, `TELEMETRY_PRIV_KEY`, or the derived accounts; only public values sourced from the beacon state are passed to the library.
- **Exception handling is correct but relies on non-obvious library behaviour.** Empirical probing confirmed that the `try` / `except` clause covers every exception the bindings raise on realistic input, and that a malformed deposit cannot interrupt iteration over the remaining keys. However, the SWIG output typemap converts every non-success return into a raised exception - including ordinary verification failure - so the equality comparison is effectively unreachable in the negative case and the `except` clause carries the entire negative path.
- **The library accepts a broader input encoding than its predecessor.** The previous implementation rejected uncompressed point encodings on length; the replacement accepts both compressed and uncompressed forms. The deposit fields are typed as unvalidated hex strings in the consensus-layer provider, so an explicit length check would restore the guarantee that was previously implicit.
- **The accounting module changes are correct and the added logging is accurate.** The counters report the values their messages describe, the per-stage instrumentation allows a divergence to be attributed to a specific layer, and the newly added warning distinguishes signature-invalid rejections from suspected front-running.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Lido Oracle
TYPE	Python
PLATFORM	EVM
TIMELINE	25.07.2026 - 26.07.2026

Scope of Audit

`src/services/deposit_signature_verification.py` `src/web3py/extensions/lido_validators.py`
`src/modules/oracles/accounting/accounting.py` `src/providers/keys/client.py`
`src/providers/http_provider.py` `scripts/build_blst.sh` `stubs/blst/__init__.pyi`

Versions Log

DATE	COMMIT HASH	NOTE
26.07.2026	579a65f4149ee829e31de708a8f8209b4aef289c	Initial Commit
26.07.2026	399a1c9e3ac6cd8a91337d84762319abc67d4340	Commit with Updates

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

[1 INTRODUCTION]

1.6 Summary of Findings

Findings Count

0	0	0	0	0
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

NO LOW SEVERITY FINDINGS

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io